Guide 2 · Codebase Walkthrough

The code in reading order, one image traced end to end, and every known issue

AgriAgent Final Year Project · 11 October 2026

> **How to read this.** Open each file as you reach its section. Line numbers refer to commit `58ba6b0` on `main` (11 October 2026); they may drift slightly after later edits. The whole project is about 7,700 lines of Python, of which you need to understand perhaps 1,500 well.

1 The map

```
src/state.py              the data that flows between stages        READ FIRST
src/workflow.py           the pipeline: calls every stage in order   READ SECOND
src/vision/               grounding_dino.py, sam2_wrapper.py, refinement.py,
visualization.py
src/agents/               deterministic_router.py (safety gate), clef_flash_router.py,
router_contract.py
src/agronomy/             system2_node.py, vlm_critic.py, knowledge_store.py,
prescription.py, providers/
src/evaluation/           iou_dice.py, benchmark_runner.py, ablation_runner.py
app/                      app.py (Streamlit page), components/pipeline.py (run +
GeoJSON), components/ui.py
tests/                    unit tests; tests/e2e/ end-to-end tests
config/  data/  scripts/  notebooks/
```

Ownership: Track 1 owns `src/vision`, `src/agents`, `src/workflow.py`; Track 2 `src/agronomy`; Track 3 `src/evaluation`, `data`; Track 4 `app`, `.github`, `tests/e2e`. Everyone must understand `state.py` and `workflow.py`.

2 The data: `src/state.py`

Everything the pipeline knows lives in one dictionary called `GraphState`. Each stage reads some keys and writes new ones. Learn these four shapes:

Type	Contents
`Box`	`x1, y1, x2, y2` (pixels), `label` (the matched prompt phrase), `score` (0–1). Made by Grounding DINO.
`Mask`	`mask_array` (H × W true/false), `predicted_iou` (SAM 2's self-estimate), the source `box`, and `label`. Made by SAM 2.
`Artifact`	A named output object with `metadata` and a `payload`. The spray map artifact's payload holds `composite_mask` and `approved_spray_zones`.
`GraphState`	Input (`image`, `text_prompt`, `field_metadata`), vision outputs (`boxes`, `masks`, `refinement_count`), reasoning outputs (`system2_needed`, `vlm_reasoning`, `prescription_report`), and `spray_map`.

Why a single shared dictionary? It makes every stage a function `state → updates`, which is easy to test and is exactly the format the LangGraph library expects (see `create_langgraph_workflow`).

3 One image, end to end: `src/workflow.py`

Open `VisionWorkflowRunner.run()` (line 385). It calls six steps in order. Trace them with the file open.

3.1 Step 1 · `detect_node` (line 97)

Reads `image` and `text_prompt` (default `"weed . grass . pigweed ."`). Calls `detector.detect_to_state_boxes()`. Writes `boxes`. If there is no image it writes `error` and `run()` stops early (line 405).

3.2 Step 2 · `segment_node` (line 108)

Sends every box to SAM 2 (`segment_to_state_masks`). Writes `masks`, one per box, each carrying its box's label.

3.3 Step 3 · refinement loop (lines 413–420, `refine_node` line 124)

Up to `max_refinements = 2` times: if any mask's `predicted_iou < 0.85`, compute a corrective point (`get_refinement_point` in `refinement.py`) and ask SAM 2 again with it (`refine_with_point`).

3.4 Step 4 · `spray_map_node` (line 180) — the most important function

This is where the spray decision happens. Read it slowly.

1. **Label policy (lines 191–232).** Each mask's label is lower-cased. It is **disallowed** if it is in a crop list (`crop`, `cotton`, `sugarbeet`, `plant`, `soil`, …) or **contains** a crop word (`cotton`, `beet`, `crop`, …). It is a **recognised weed** only if it is not disallowed and contains a weed word (`weed`, `grass`, `pigweed`, …). Anything else is skipped. **Fail-closed**: unknown labels are not sprayed.
2. **Composite mask (lines 234–252).** Approved masks are OR-ed into one `composite_mask`, and each is stored as an approved zone with its own mask.
3. **Infestation (line 261).** Weed pixels divided by image pixels, as a percentage.
4. **Safety gate (lines 263–272).** `DeterministicRouter().route(...)`. If infestation exceeds 85% the action is `SUPPRESS_SPRAY`, and the composite mask and zones are wiped.
5. **Savings (line 274).** `compute_herbicide_savings` grows the mask by a buffer and computes 1 − sprayed / total.
6. **Output (lines 278–316).** `spray_map` (numbers for the dashboard) and the `spray_map` artifact (the masks, used by the GeoJSON export).

3.5 Step 5 · `triage_node` (line 318)

For each box: ask the deterministic router whether to escalate (low confidence below 0.70, poor mask quality, ambiguity). If the label is not a known weed or crop word, it is **ambiguous**, and the Clef Flash router is consulted with a crop of the image. Writes `system2_needed`.

3.6 Step 6 · `system2_node` (line 375)

Only if `system2_needed`: runs `System2Reasoner` (`src/agronomy/system2_node.py`) for diagnosis and prescription.

> **Notice the order.** The spray map is built in step 4. Escalation is decided in step 5. Nothing in step 5 or 6 withdraws the spray map. This is known issue 2 (Section 10).

4 Vision code

4.1 `src/vision/grounding_dino.py`

- `GroundingDINOEngine.__init__` (line 71) and `_initialize_model` (line 91): loads the model with Hugging Face `transformers`, on GPU if available.
- **Lines 106–118: if anything fails while loading, it quietly sets `mock_mode = True`.** Known issue 5.
- `_generate_mock_detections` (line 136): three hard-coded boxes at fixed relative positions, all labelled with the **first phrase of the prompt**. This is why mock output always shows three boxes.

- `detect` (line 177): the real path. Prepares inputs, runs the model under `torch.no_grad()` (no training), post-processes into boxes with the two thresholds (lines 225–239, written to work with old and new `transformers` versions).

4.2 `src/vision/sam2_wrapper.py`

- `_initialize_model` (line 89): tries Meta's `sam2` package first, then the `transformers` version. It also falls back to mock silently on failure (lines 125, 131).
- `segment_boxes` (line 207): one mask per box, `multimask_output=False` (one answer, not three candidates), score taken as `predicted_iou`.
- `refine_with_point` (line 296): re-prompt with box + corrective point. **In mock mode it simply adds 0.08 to the IoU (line 333).** Known issue 6.

4.3 `src/vision/refinement.py`

Pure NumPy. `normalize_box_coordinates` makes any box safe (sorted, rounded, clamped). Morphology helpers (lines 71–135). `compute_error_centroid` (line 137) picks the corrective point. `get_refinement_point` (line 344) is what the workflow calls: returns `None` if the mask is good enough.

5 The safety layer

5.1 `src/agents/router_contract.py`

Defines the four possible actions: `ACCEPT`, `REFINE`, `ESCALATE_SYSTEM2`, `SUPPRESS_SPRAY`, and the `RouterDecision` object every router returns (action, human-readable reason, latency, backend). Any router must implement `route()`. This is the **interface** that lets us swap routers.

5.2 `src/agents/deterministic_router.py` — memorise this

`route()` checks rules **in this order** and returns at the first match:

#	Rule	Action
1	infestation > 85% (line 55)	`SUPPRESS_SPRAY`
2	predicted IoU < 0.85 (line 76)	`REFINE`
3	species ambiguity flag set (line 94)	`ESCALATE_SYSTEM2`
4	detection confidence < 0.70 (line 111)	`ESCALATE_SYSTEM2`
5	predicted IoU < 0.80 (line 128)	`ESCALATE_SYSTEM2`
6	otherwise	`ACCEPT`

Order matters: safety first, then quality, then uncertainty. Rule 5 can never fire after rule 2, because anything below 0.80 is already below 0.85. Be ready to say so if asked; it is harmless but redundant.

5.3 `src/agents/clef_flash_router.py`

Runs the deterministic checks **first** (line 141); only if those say `ACCEPT` does it ask Clef. If the model cannot load (our case: no 24 GB+ GPU), it returns `ESCALATE_SYSTEM2` (line 168): **it fails safe, never towards spraying.** In mock mode it simply echoes the ambiguity flag. [Being rewritten in Week 4 (schedule task T2.8) to call Clef Flash over HTTP, on Cloudflare or a local `llama-server`; the fail-safe rule stays.]

6 Agronomy (System 2)

File	Role
`system2_node.py`	Glue: image → `VLMCritic.diagnose` → weed + stage → `get_herbicide_prescription` → report artifact.

`vlm_critic.py`	Builds the "expert agronomist, reply only in JSON" prompt, sends it through a provider, validates the reply with Pydantic. Mock returns a fixed diagnosis.
`providers/`	`local_qwen.py` (local OpenAI-compatible server, `Qwen/Qwen3.5-4B`), `hosted_groq.py` (Groq API, `qwen/qwen3.8-27b`). Both implement `provider_contract.py`.
`knowledge_store.py`	Loads the ICAR bulletin PDF plus hand-transcribed table rows (`CANONICAL_ICAR_TABLE_ROWS`, line 121), embeds them, answers queries. ChromaDB in production, in-memory for tests.
`prescription.py`	The safety logic for chemicals. See below.

`create_chemical_prescription` (line 189), in order:

1. If the weed query itself names a banned chemical, return `unapproved_chemical_flagged`.
2. Retrieve matching knowledge; drop weak matches.
3. **No match → `no_knowledge_found`, and no chemical is suggested.** No hallucinated advice.
4. Extract chemical, dose and water volume from the retrieved row.
5. If that chemical is banned (`BANNED_HERBICIDES`, line 30) → flagged.
6. If the dose exceeds the statutory ceiling (`STATUTORY_DOSAGE_CEILINGS_G_PER_HA`, line 43) → clamp it and add a warning.
7. Attach toxicity band and the mandatory disclaimer. Status `ok`.

The banned list and ceilings were transcribed into code from the bulletin's tables. They must be checked against the PDF by hand (Track 2 task).

7 The app

- `app/app.py` · `main()` (line 105): sidebar → upload or sample image → `run_pipeline_on_image` → KPI cards → overlays → prescription → GeoJSON download.
- `app/components/ui.py` · `render_sidebar` (line 28): **Mock Mode is ticked by default.** Untick it for real models.
- `app/components/pipeline.py`:
  - `execute_agriagent_workflow` (line 213): builds the runner and runs it. If "Force System 2" is ticked, also runs the prescription step; if no weed name is known it falls back to `"Trianthema"`.
  - `build_spray_geojson` (line 381): **the safety-fixed exporter.** Reads only `approved_spray_zones` from the artifact; empty if suppressed. Converts each zone mask to polygons and places them at a fixed coordinate near Mumbai (line 400).

8 Evaluation

- `src/evaluation/iou_dice.py`: `compute_iou`, `compute_dice`, `evaluate_batch` (with 95% confidence interval), `compute_herbicide_savings`.
- `data/manifests/eval_manifest_v1.json`: **20 sugar-beet images** from `tests/fixtures/sample/` with human annotations for both **crop** and **weed** boxes. This means a first real evaluation, including how much crop ends up on the spray map, can be run immediately with real weights.
- `benchmark_runner.py`: runs the pipeline over the manifest and scores it against annotations.
- `ablation_runner.py`: experiments A–E (with/without refinement, parameter sweeps, routers). So far only ever run in mock mode.

9 Tests

File	Proves

`test_spray_export_safety.py`	Crop labels, `cropweed`, over-85% infestation and raw boxes never reach GeoJSON. **Know this one well.**
`test_routers.py`	Each deterministic rule; Clef fails safe.
`test_prescription_safety.py`	Banned chemicals flagged, dose clamped, no advice without evidence.
`test_vision_pipeline.py`, `test_refinement.py`	Wrappers, workflow order, refinement maths and edge cases.
`test_agronomy.py`, `test_providers.py`	Knowledge store, JSON parsing, providers (with mocked network).
`test_app.py`, `test_visualization.py`	App helpers, image decoding, overlays.
`test_metrics.py`, `test_benchmark_runner.py`, `test_ablations.py`	Metric formulas and evaluation harness.
`tests/e2e/` (4 tiers)	Whole pipeline: features, edge cases, combinations, realistic scenarios.

All tests run in mock mode. They prove the **logic**; they do not prove the **models** are accurate.

10 Known issues

Name these yourself before anyone else does. Each is also a concrete improvement for the final weeks.

ID	Issue and location	Severity
K1	Gate called with confidence and IoU hard-coded to `1.0` (`workflow.py:264–265`). Only the 85% rule applies to the final map; a low-confidence weed is still sprayed.	high
K2	Triage (step 5) runs after the spray map (step 4) and never withdraws it.	high
K3	Weed/crop labels are just the matched prompt words. Nothing visually distinguishes weed from crop. The core research problem.	high
K4	`scripts/run_live_pipeline.py` default prompt `"cotton weed . broadleaf plant ."` is rejected by the crop rule. Verified: 3 weeds found, 0% sprayed, status still says “Spray Prescription Generated”.	medium
K5	Vision wrappers silently switch to mock if weights fail to load (`grounding_dino.py:106–118`, `sam2_wrapper.py:125–131`). A “live” demo could show mock boxes without warning.	high for demo
K6	Mock refinement adds a fixed +0.08 to IoU (`sam2_wrapper.py:333`). Any mock result “showing refinement helps” is built into the mock.	medium
K7	The app’s confidence slider filters only what is drawn (`app.py:88–93`), not the spray map or GeoJSON.	medium

K8	GeoJSON falls back to `Pyrithiobac-sodium` 75 g/ha when no prescription exists (`pipeline.py:434`), and the forced System 2 path falls back to weed `"Trianthema"` (line 279–281). Hard-coded advice contradicts "no advice without evidence".	medium
K9	GeoJSON coordinates are fake: fixed point plus pixel scale (`pipeline.py:400`).	known limit
K10	`compute_herbicide_savings` uses a different buffer size with and without OpenCV (`iou_dice.py:64` vs `71`), so savings depend on what is installed.	low
K11	Speed claims in comments ("< 40 ms", "< 35 ms") were never measured.	claim
K12	Banned list and dose ceilings in `prescription.py` are hand-transcribed and unchecked against the PDF.	medium

11 Self-check

1. Name the six steps of `run()` in order and the key each one writes.
2. A mask has label `"cotton weed"`. Is it sprayed? Why? Which lines decide?
3. A mask has label `"thistle"`. Is it sprayed? Is it escalated?
4. Infestation is 90%. What happens to the spray map and the GeoJSON?
5. List the deterministic router's rules in order. Why does the order matter?
6. What does the Clef router do when the model cannot load? Why is that the right choice?
7. In `create_chemical_prescription`, what happens when no relevant knowledge is found?
8. Where exactly does the GeoJSON exporter get its polygons from, and why not from the boxes?
9. How would you tell, during a demo, whether you are seeing real or mock output?
10. Pick known issue 1 or 2 and describe the fix in plain words.